

LAPORAN PRAKTIKUM KEAMANAN INFORMASI

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah praktikum keamanan informasi

Semester 5 Tahun Akademik 2025/2026

Oleh:

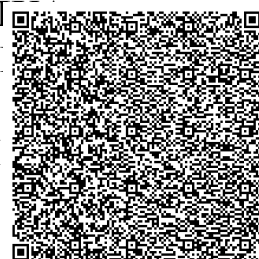
Nama : Kamila Zahra R.

NPM : 247006111161

Kelas : EF



**LABORATORIUM INFORMATIKA
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SILIWANGI
TASIKMALAYA
2026**



BAB I

PENDAHULUAN

1. Latar Belakang

Perkembangan aplikasi berbasis web yang sangat cepat diikuti oleh semakin meningkatnya ancaman keamanan siber. Kekurangan keamanan dalam aplikasi web bisa digunakan oleh orang yang tidak berhak untuk mencuri informasi rahasia, mengubah izin akses pengguna, hingga mengontrol sistem secara penuh. Oleh karena itu, pengujian penetrasi menjadi cara penting untuk menemukan, menganalisis, dan mengurangi celah keamanan sistem sebelum dimanfaatkan oleh pihak-pihak yang tidak bertanggung jawab.

Dalam dunia akademik, pengujian penetrasi secara etis dilakukan di lingkungan laboratorium yang terisolasi, seperti aplikasi web yang rentan Damn Vulnerable Web Application (DVWA). Dengan modul praktikum ini, kita melakukan pengujian terhadap berbagai skenario kerentanan pada website, seperti SQL Injection, Cross-Site Scripting (XSS), Command Injection, Brute Force, Cross-Site Request Forgery (CSRF), Unrestricted File Upload, dan XSS-Keylogger.

2. Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar dan cara melakukan pengujian penetrasi di aplikasi web.
2. Menganalisis cara kerja celah kerentanan seperti SQL Injection, XSS, Command Injection, Brute Force, CSRF, File Upload, dan Keylogger.
3. Menerapkan langkah-langkah uji keamanan dengan menggunakan alat-alat standar seperti Kali Linux dan Burp Suite.
4. Membuat saran penanggulangan dan tindakan perlindungan yang tepat untuk menutupi celah kerentanan dalam aplikasi web.

3. Manfaat Praktikum

Manfaat yang diperoleh setelah melakukan praktikum ini adalah :

1. **Memahami Konsep Kerentanan Aplikasi Web:**
Mahasiswa dapat memahami secara mendalam mekanisme kerja dari berbagai jenis celah keamanan utama (seperti SQL Injection, XSS, Command Injection, Brute Force, CSRF, File Upload, dan Keylogger) yang sering ditemukan pada sistem nyata.
2. **Menerapkan Metodologi Ethical Hacking:**
Memberikan pemahaman praktis mengenai alur kerja pengujian penetrasi (*penetration testing*) secara beretika dan terstruktur sesuai standar industri keamanan siber.
3. **Keterampilan Mengoperasikan Tools Keamanan:**
Mahasiswa terlatih dalam menggunakan berbagai

perangkat lunak pengujian penetrasi populer seperti Kali Linux, Burp Suite, dan utilitas terminal untuk analisis keamanan aplikasi web.

4. **Kemampuan Merumuskan Mitigasi Keamanan:** Tidak hanya sekadar menemukan celah, praktikum ini melatih mahasiswa untuk menganalisis risiko dan merumuskan langkah pengamanan (*defensive security*) yang tepat guna menutup celah kerentanan pada kode aplikasi.
5. **Peningkatan Kesadaran Keamanan (Security Awareness):** Membentuk sudut pandang seorang *security auditor* dalam melihat potensi ancaman, sehingga dapat merancang sistem yang lebih aman (*secure coding*) di masa mendatang.

BAB II

LANDASAN TEORI

1. SQL Injection (SQLi)

SQL Injection terjadi ketika input dari pengguna dimasukkan langsung ke dalam kueri basis data SQL tanpa proses sanitasi yang memadai. Hal ini memungkinkan penyerang menyisipkan perintah SQL buatan yang dapat memanipulasi logika kueri asli untuk menampilkan, mengubah, atau menghapus data.

2. Cross-Site Scripting (XSS) & DOM-Based XSS

Cross-Site Scripting (XSS) merupakan kerentanan di mana aplikasi web menyisipkan skrip sisi klien (*client-side script*) seperti JavaScript yang tidak tersaring ke dalam dokumen

web. Pada DOM-Based XSS, manipulasi terjadi langsung pada struktur *Document Object Model* (DOM) di browser pengguna tanpa melalui pemrosesan kembali di server.

3. Command Injection

Command Injection terjadi saat aplikasi web meneruskan input pengguna secara langsung ke dalam perintah shell sistem operasi (*system command*). Kerentanan ini memungkinkan penyerang menjalankan perintah lokal pada sistem server web.

4. Brute Force Attack

Serangan Brute Force adalah metode coba-coba (*trial-and-error*) yang dilakukan secara otomatis untuk menebak kombinasi kredensial (*username* dan *password*) hingga menemukan pasangan yang valid.

5. Cross-Site Request Forgery (CSRF)

CSRF adalah serangan yang memaksa pengguna terautentikasi untuk mengeksekusi aksi yang tidak diinginkan pada aplikasi web tempat mereka sedang aktif. Penyerang memanfaatkan kepercayaan yang dimiliki aplikasi terhadap sesi atau cookie pengguna.

6. Unrestricted File Upload

Kerentanan pengunggah file terjadi ketika server menerima file tanpa validasi tipe, ekstensi, atau konten file secara ketat. Mengunggah skrip yang dapat dieksekusi (seperti file PHP) ke folder yang berlokasi di web root dapat memicu eksekusi kode acak (*Remote Code Execution*).

7. XSS-Keylogger

XSS-Keylogger menggabungkan celah XSS dengan skrip perekam ketikan kunci (*keylogger*) untuk mencuri data sensitif seperti kredensial atau input rahasia pengguna yang diketikkan pada antarmuka web secara langsung.

BAB III PRAKTIKUM

1. Alat dan Bahan

No	Nama Alat / Bahan	Fungsi / Deskripsi
1.	Sistem Operasi Kali Linux	Platform pengujian penetrasi dan keamanan siber.
2.	Damn Vulnerable Web Application (DVWA)	Aplikasi web terisolasi berbasis PHP/MySQL untuk latihan

No	Nama Alat / Bahan	Fungsi / Deskripsi
		keamanan.
3.	Burp Suite Community Edition	Alat web proxy, interceptor, dan penguji serangan otomatis (Intruder).
4.	Mozilla Firefox	Browser web untuk navigasi antarmuka aplikasi.
5.	Text Editor (Mousepad / Nano)	Editor untuk menyusun skrip eksploitasi kustom (seperti `csrf.html`).

2. Langkah-Langkah Praktikum

Praktikum dilakukan melalui proses berikut:

- a. **SQL Injection:**Buka menu SQL Injection pada aplikasi DVWA.Masukkan karakter pembanding logika pada kolom input: ' OR '1'='1.Tekan Submit dan amati data kredensial seluruh pengguna yang berhasil ditampilkan.Untuk pengujian otomatis, jalankan utilitas sqlmap melalui terminal dengan perintah: sqlmap -u "http://[URL_TARGET]" --cookie="[SESSION_COOKIE]".
- b. **XSS (DOM Input pada URL):** Navigasi ke modul XSS (DOM) pada DVWA.Sisipkan payload JavaScript pada parameter URL:
<script>alert('xss berhasil!')</script>.Lakukan

pengujian pencurian informasi sesi cookie
menggunakan payload:

```
<script>alert(document.cookie)</script>.
```

- c. **Command Injection:** Buka menu Command Injection pada DVWA. Ketikkan alamat IP dasar diikuti operator perangkaian perintah: 127.0.0.1 & dir. Coba akses direktori konfigurasi dengan perintah: 127.0.0.1 & dir ..\..\config. Tampilkan isi file konfigurasi sensitif dengan perintah: 127.0.0.1 & type ..\..\config\config.inc.php.

- d. **Brute Force Attack:** Buka Burp Suite dan aktifkan fungsi Intercept pada menu Proxy. Lakukan upaya login pada modul Brute Force DVWA. Kirimkan request hasil intercept ke modul Intruder. Atur variabel posisi serangan pada parameter username dan password dengan tipe serangan **Cluster Bomb**. Konfigurasikan daftarnya pada menu Payload (Simple List). Tambahkan kata penanda kegagalan pada bagian Options (Grep - Match), misalnya kata incorrect. Jalankan serangan (Start Attack) dan analisis panjang respons HTTP untuk menemukan kredensial yang valid.

- e. **Cross-Site Request Forgery (CSRF):** Buka modul CSRF DVWA dan amati struktur form dengan mengklik tombol **View Source**. Buka text editor (Mousepad) dan buat file HTML kustom bernama csrf.html. Salin struktur form ubah password dan atur nilai (value) password baru yang diinginkan. Sesuaikan atribut action agar mengarah langsung ke URL target DVWA: <http://127.0.0.1:42001/vulnerabilities/csrf/>. Tam

bahkan perintah pemanggilan otomatis `document.forms[0].submit()`. Buka file `csrf.html` pada browser yang sedang memiliki sesi aktif DVWA untuk mengeksekusi perubahan password.

- f. **Unrestricted File Upload & Analysis:** Amati mekanisme pengunggahan file pada modul File Upload DVWA. Lakukan analisis teoritis terhadap risiko pengunggahan file skrip tanpa pembatasan ekstensi. Identifikasi jalur akses file yang diunggah pada direktori target (misalnya `/hackable/uploads/`).
- g. **XSS-Keylogger & Advanced Monitoring:** Analisis skrip pemantauan antarmuka pada lingkungan pengujian eksternal. Amati simulasi pengiriman instruksi modifikasi tampilan (misalnya latar belakang) dan pop-up peringatan dari jarak jauh.

BAB IV

HASIL PRAKTIKUM & ANALISIS

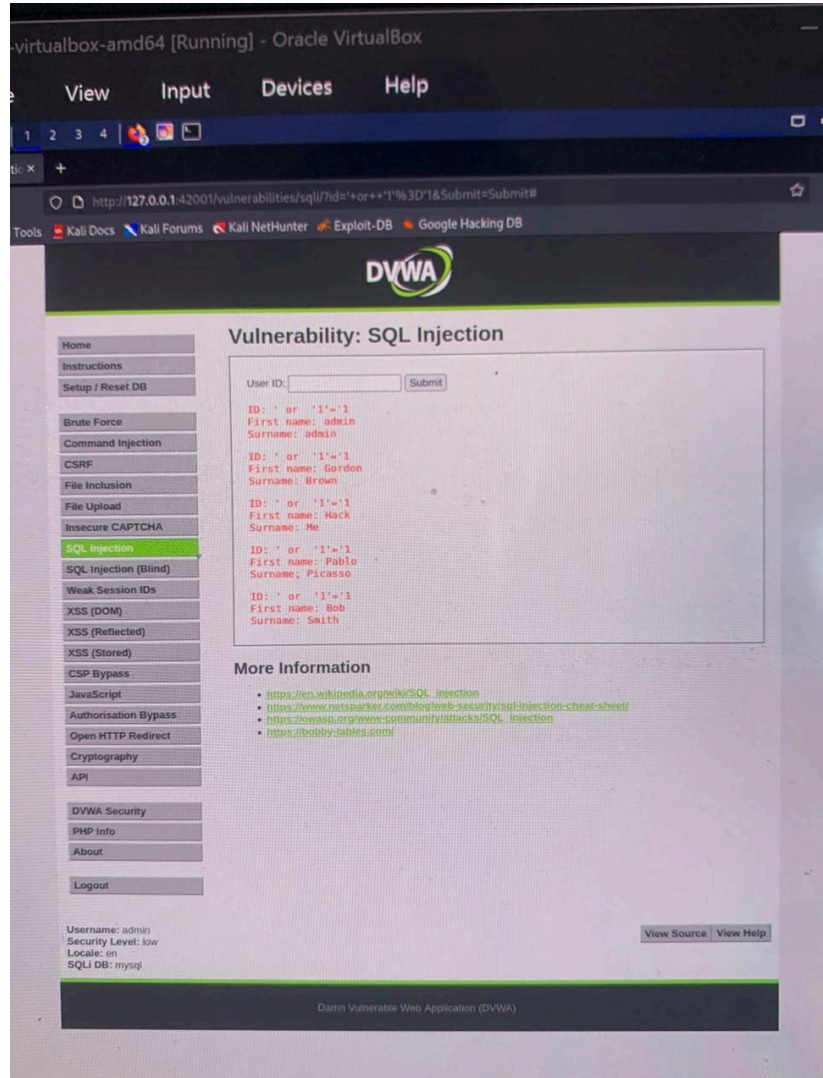
1. Hasil Praktikum

Berdasarkan langkah-langkah yang telah dilaksanakan, berikut adalah dokumentasi hasil praktikum:

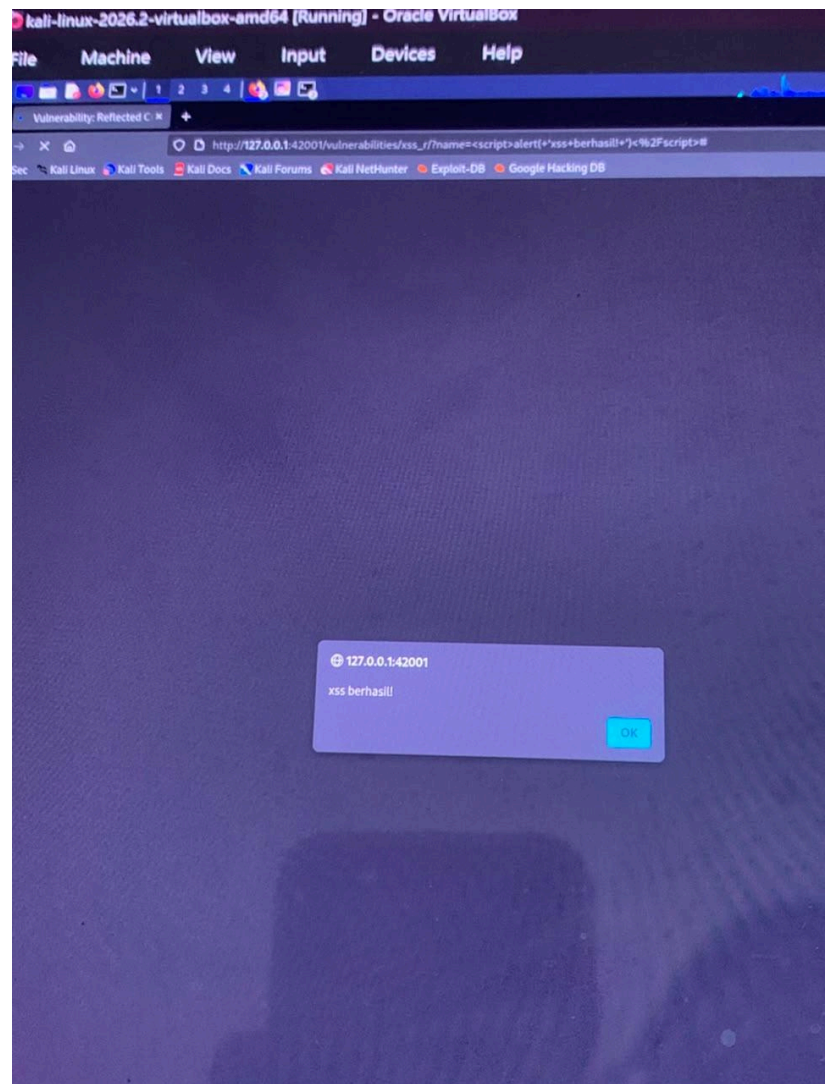
a. Pengujian SQL Injection

Tampilan halaman *Vulnerability: SQL Injection* di DVWA setelah memasukkan payload `' OR '1'='1` [cite: 1, 2]. Hasilnya menampilkan daftar lengkap ID, First name, dan Surname dari

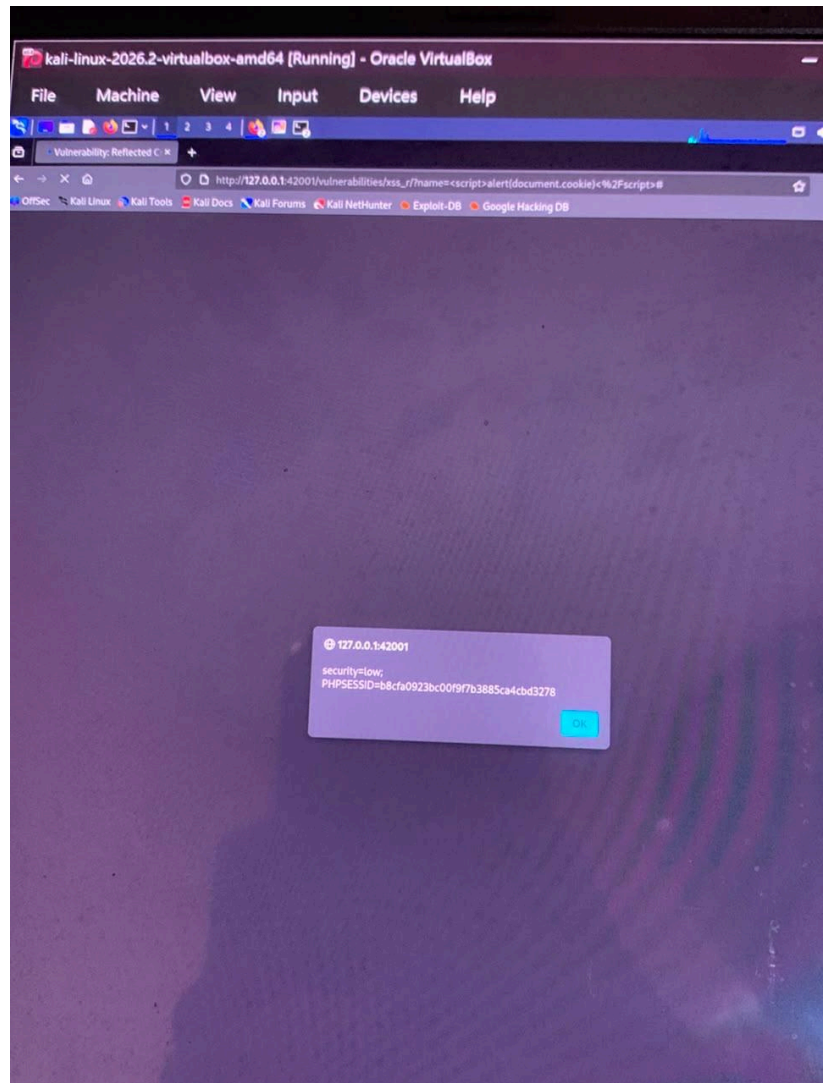
seluruh user (admin, Gordon, Hack, Pablo, Bob, dll.).



b. Pengujian Cross-Site Scripting (XSS)



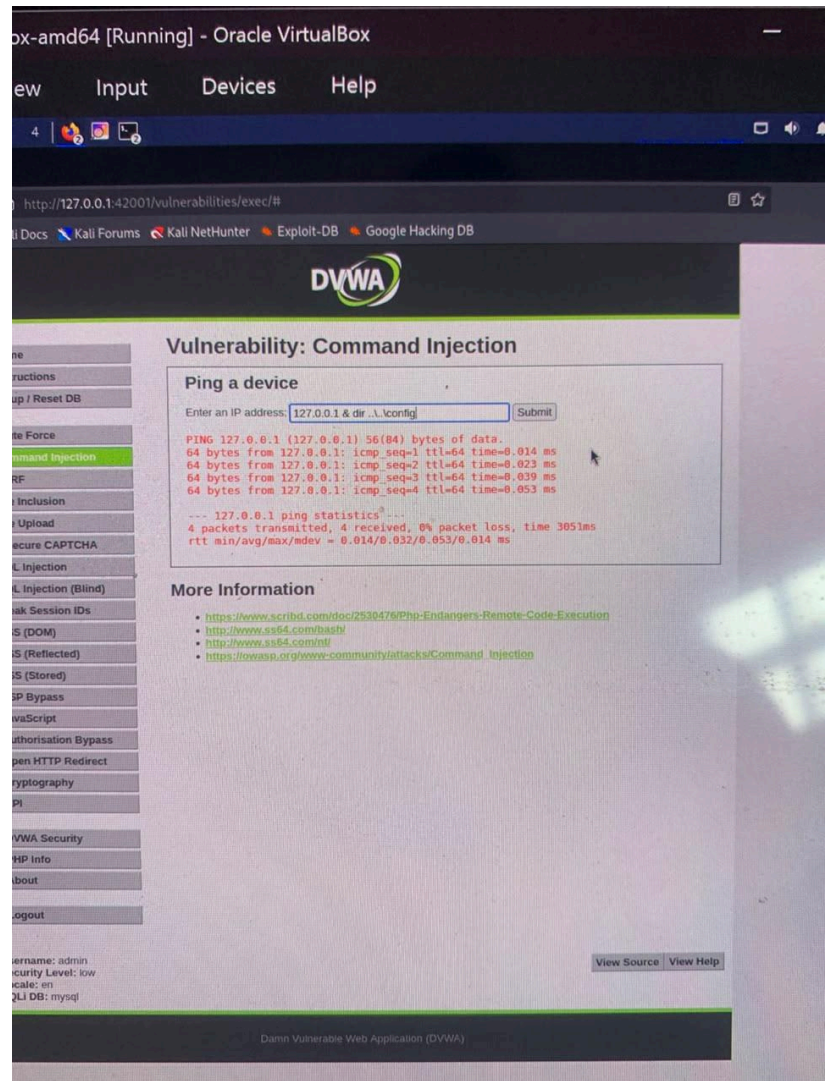
Menunjukkan *pop-up alert* browser bertuliskan **xss berhasil!**[cite: 1, 2] hasil dari injeksi script `<script>alert('xss berhasil!')</script>`



Menunjukkan *pop-up alert* yang berhasil menampilkan data **Cookie/Sesi aktif** (PHPSESSID=b8cfa...) hasil dari payload `<script>alert(document.cookie)</script>`.

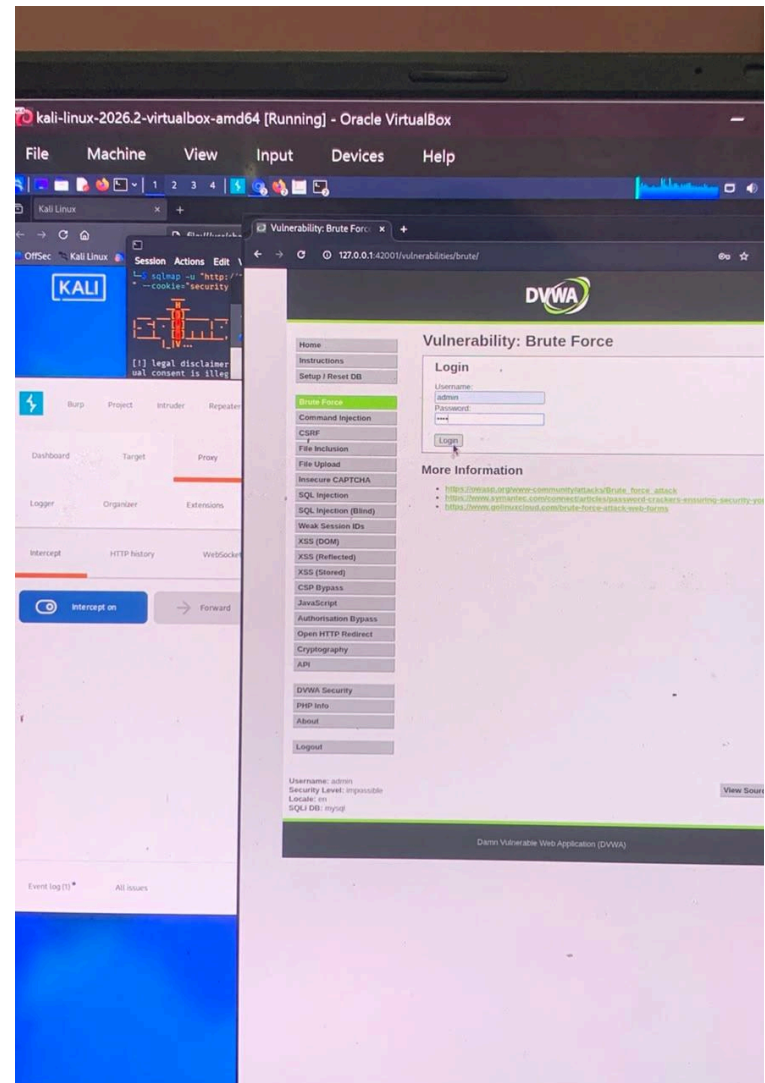
c. Pengujian Command Injection

Tampilan modul *Command Injection* saat mengeksekusi perintah 127.0.0.1 & dir ..\..\config pada kolom input ping. Output memperlihatkan statistik balasan ping yang dibarengi dengan eksekusi perintah direktori.



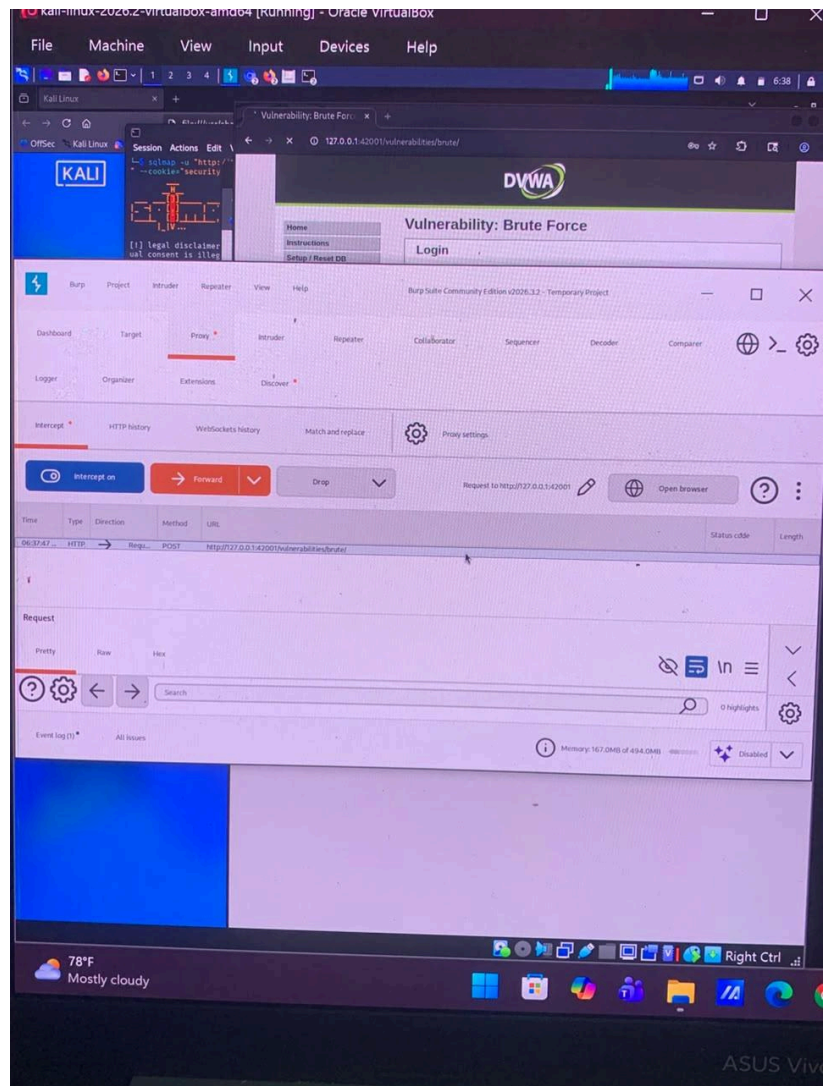
d. Modul Brute Force (Proses Intercept & Burp Suite)

- Upaya Login & Intercept



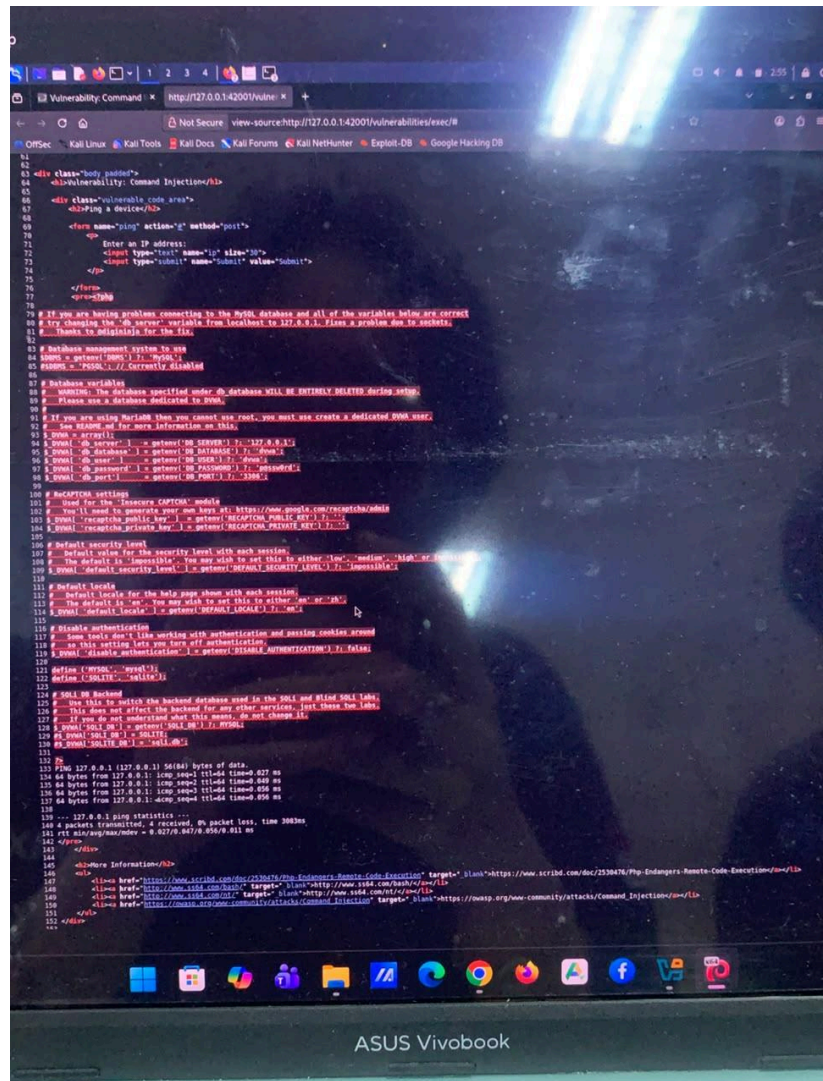
Tampilan saat melakukan upaya login pada modul *Vulnerability: Brute Force* DVWA dengan fitur **Intercept ON** di Burp Suite.

- Hasil Capturing Request



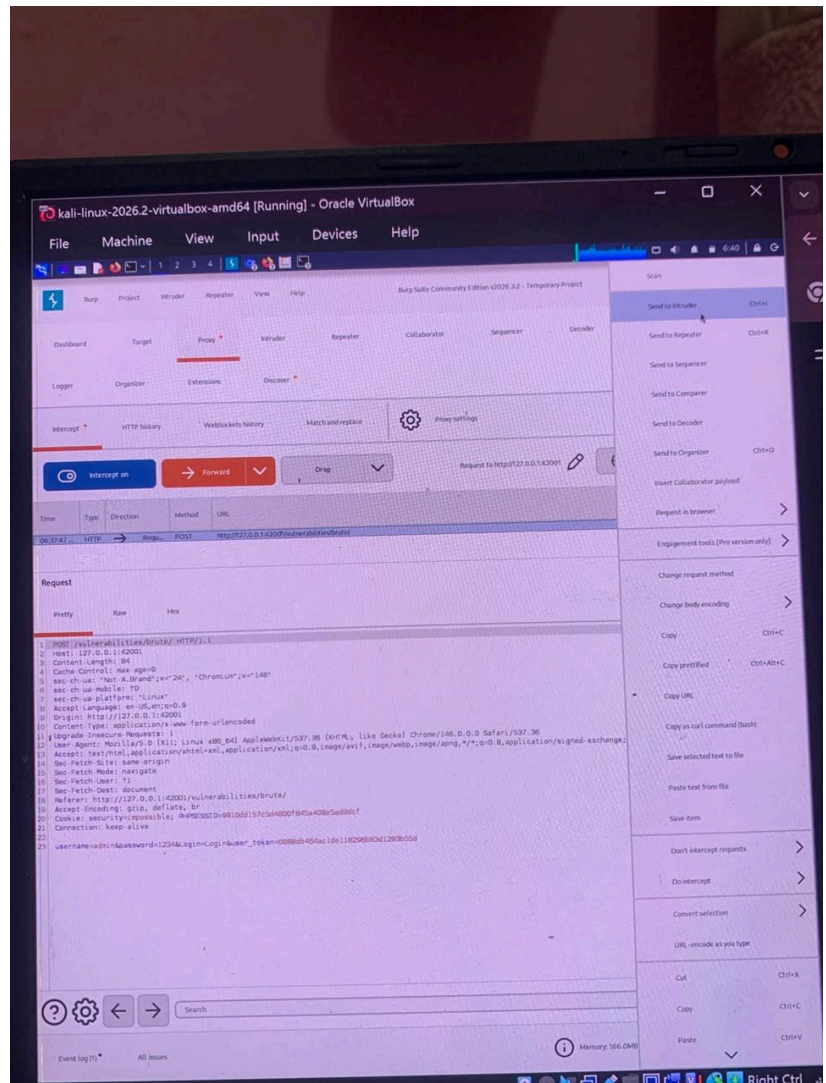
Tampilan Burp Suite Proxy Intercept yang berhasil menangkap (*capture*) lalu lintas request HTTP POST ke URL target vulnerabilities/brute/ sebelum dikirimkan ke modul **Intruder**.

- Pendukung Analisis Command Injection (Source Code View)

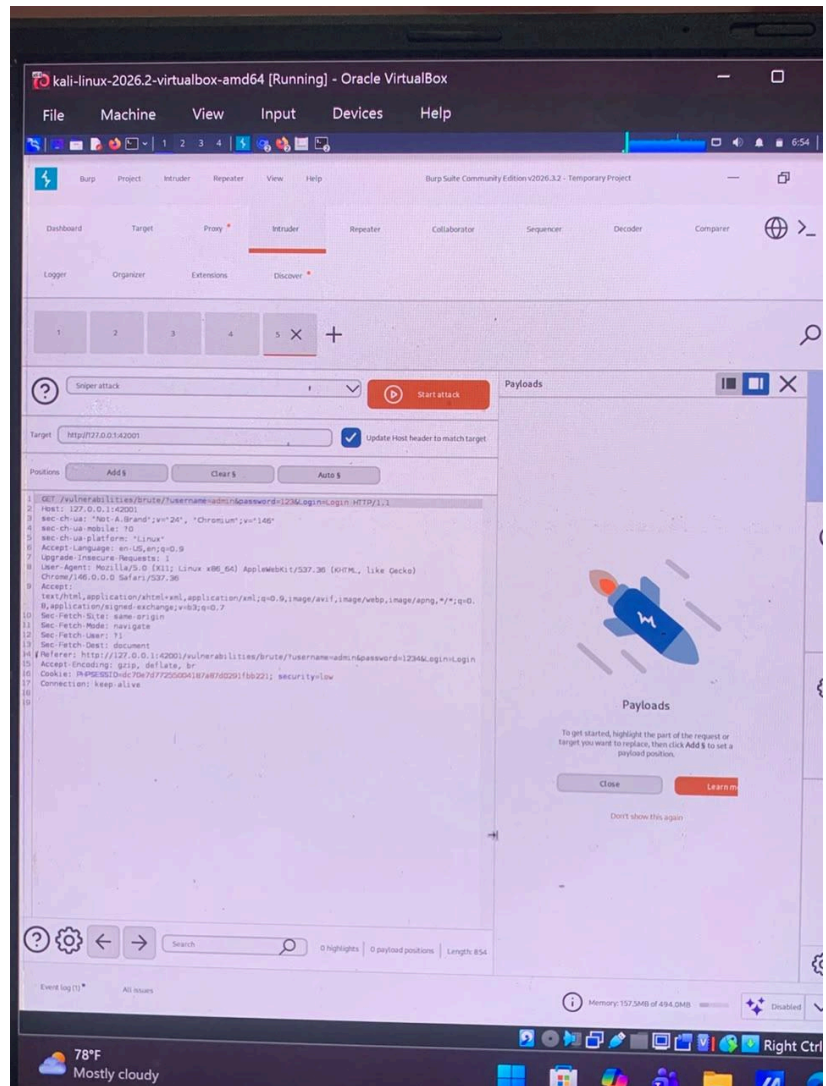


Tampilan *View* *Source*
 (view-source:[<http://127.0.0.1:42001/vulnerabilities/exec/#>])(<http://127.0.0.1:42001/vulnerabilities/exec/#>)) yang menunjukkan kode sumber PHP DVWA. Foto ini memperlihatkan secara jelas bahwa aplikasi menggunakan fungsi `shell_exec` atau perintah langsung tanpa sanitasi input, yang memperkuat analisis teoritis celah kerentanan *Command Injection*.

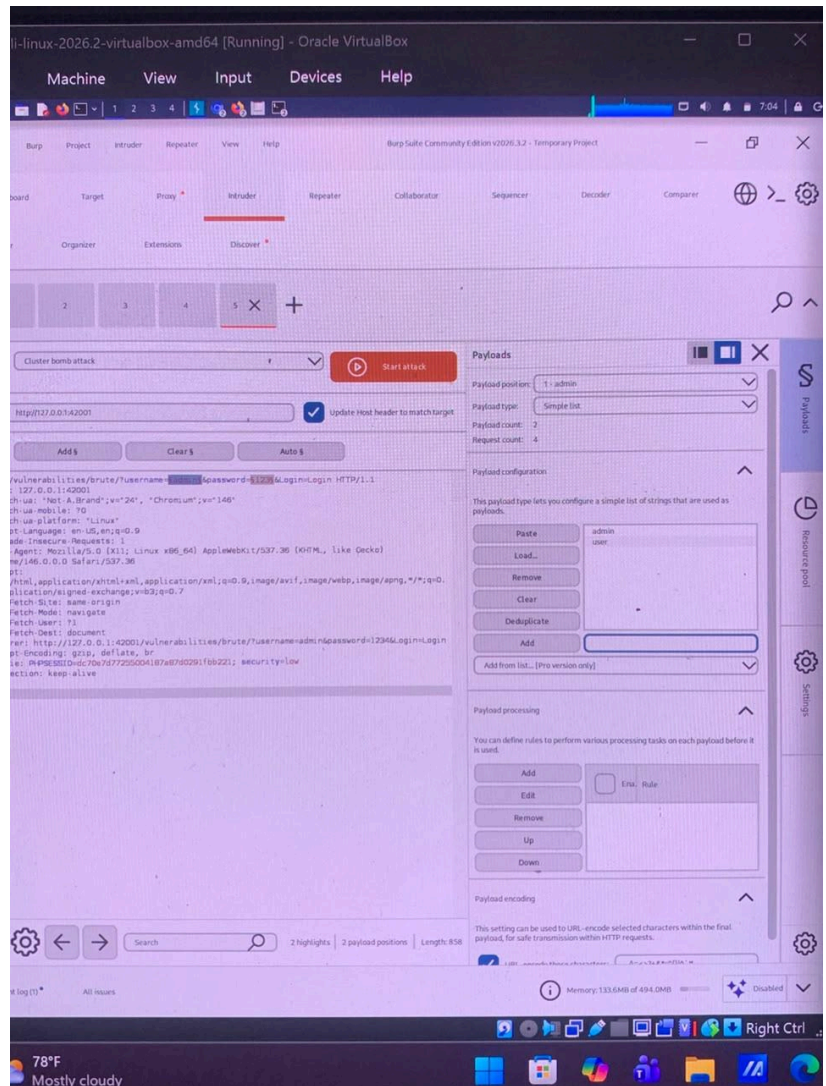
- Detail Alur Serangan Brute Force (Burp Suite Intruder)

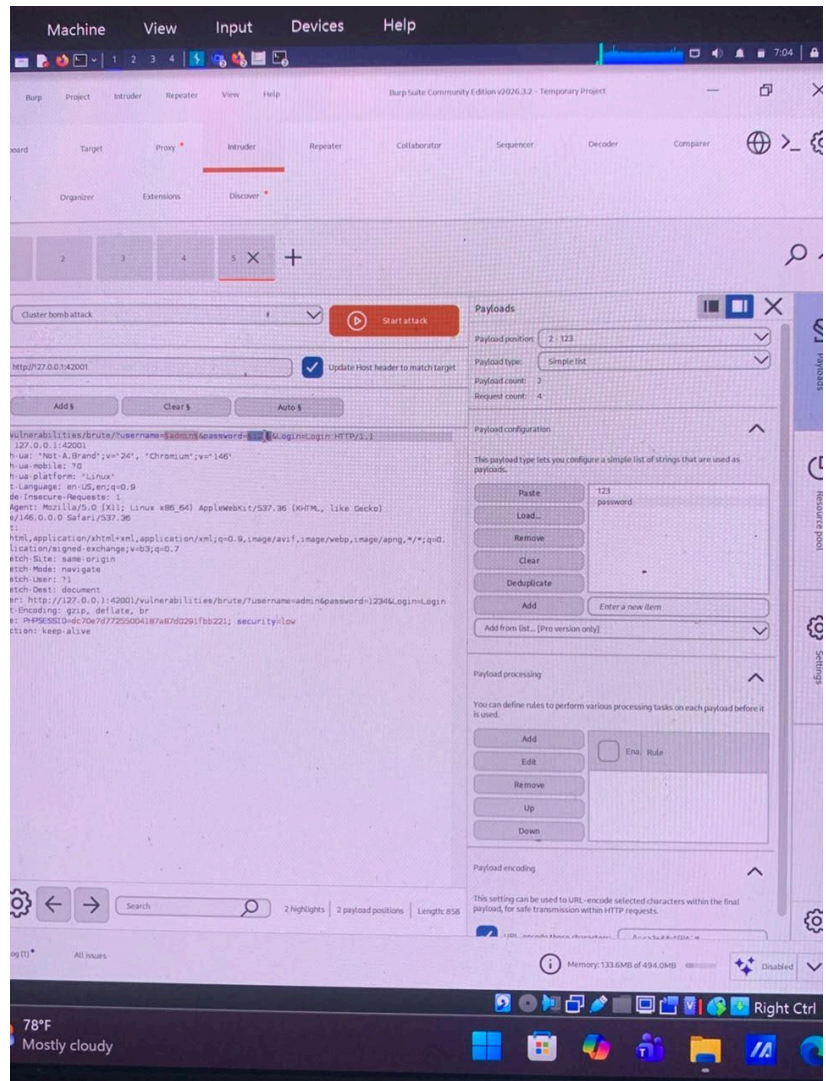


Menunjukkan menu klik kanan pada Burp Suite Proxy untuk melakukan **Send to Intruder**.

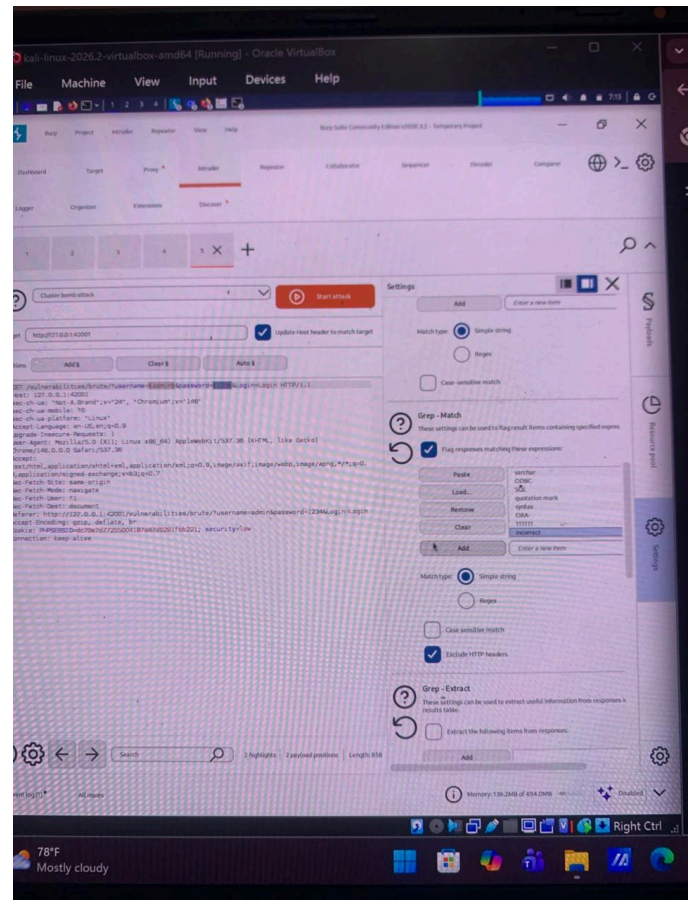


Menunjukkan pemilihan jenis serangan **Cluster Bomb** dan penentuan posisi variabel payload.

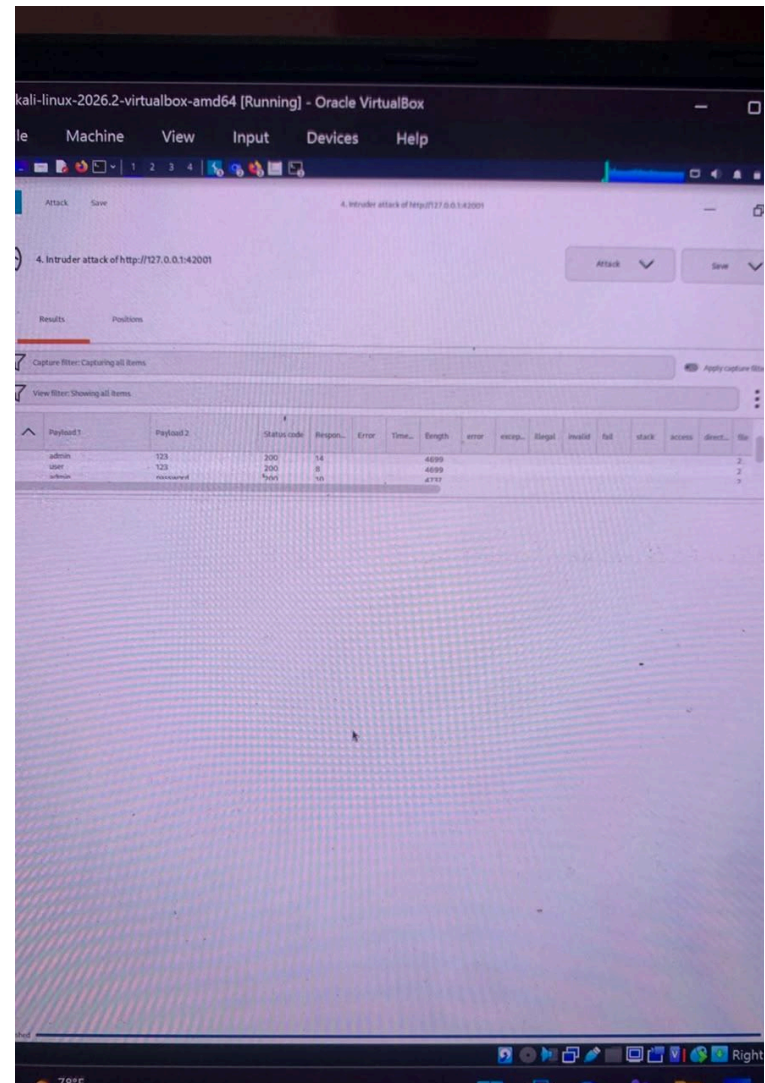




Menunjukkan pengisian daftar kata kunci (Payloads) untuk username (admin, user) dan password (123, password).

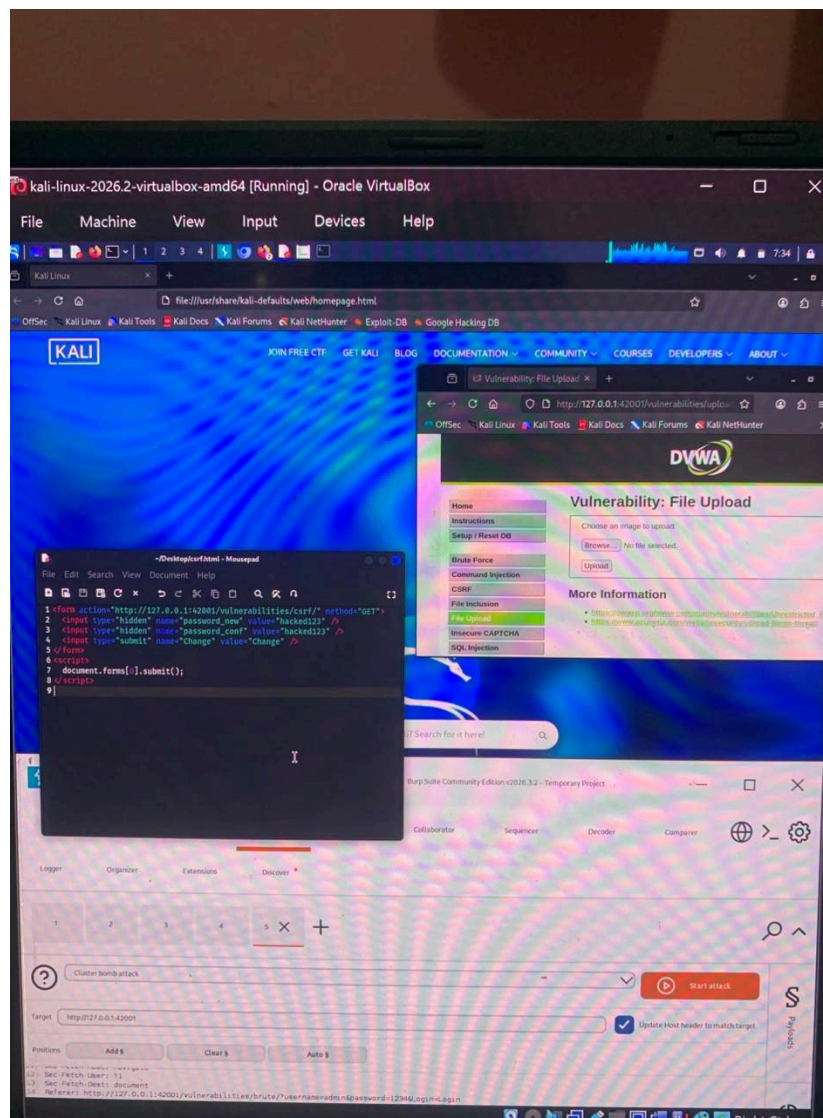


Menunjukkan pengaturan **Grep - Match** dengan menambahkan flag kata incorrect untuk membedakan login sukses dan gagal.



Menunjukkan tabel **Attack Results** di Intruder tempat ditemukannya kombinasi kredensial yang valid ditandai dengan perbedaan nilai *Length* respons HTTP.

e. Pembuatan Skrip CSRF di Text Editor

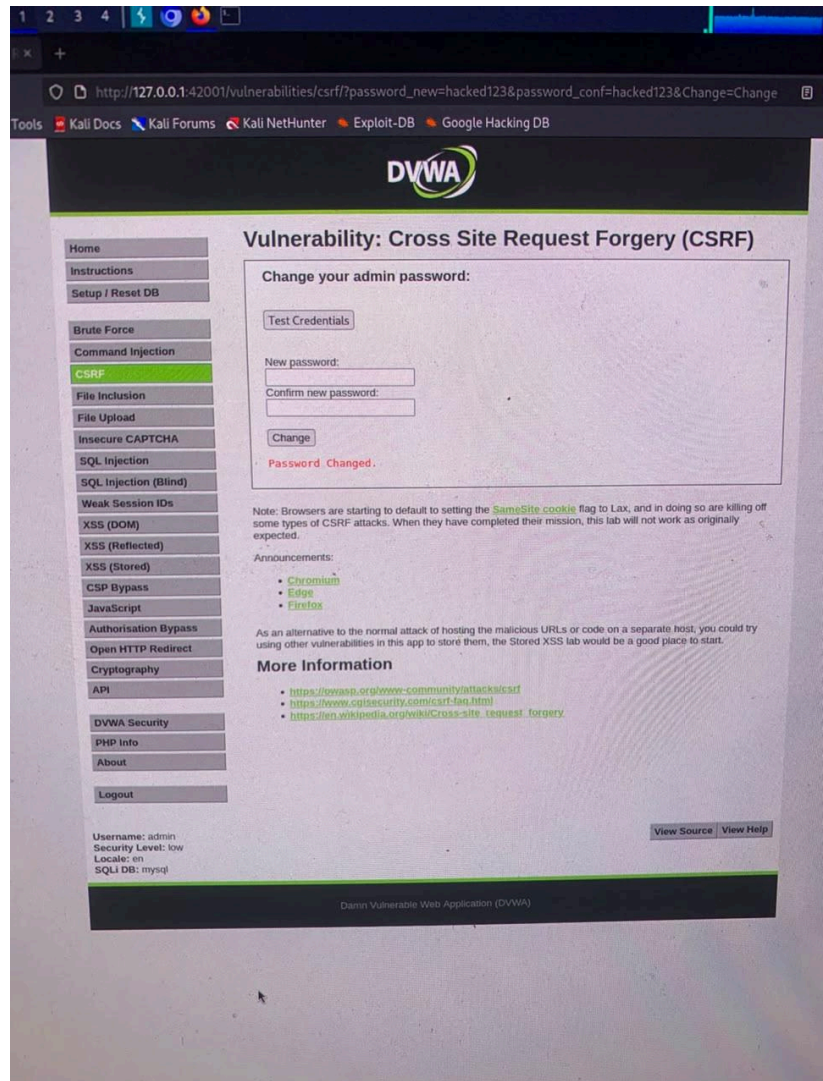


Menunjukkan koding skrip csrf.html menggunakan **Mousepad** (dengan parameter password_new, password_conf, dan document.forms[0].submit()) yang dibuka bersandingan dengan halaman DVWA.

f. Hasil Eksploitasi CSRF

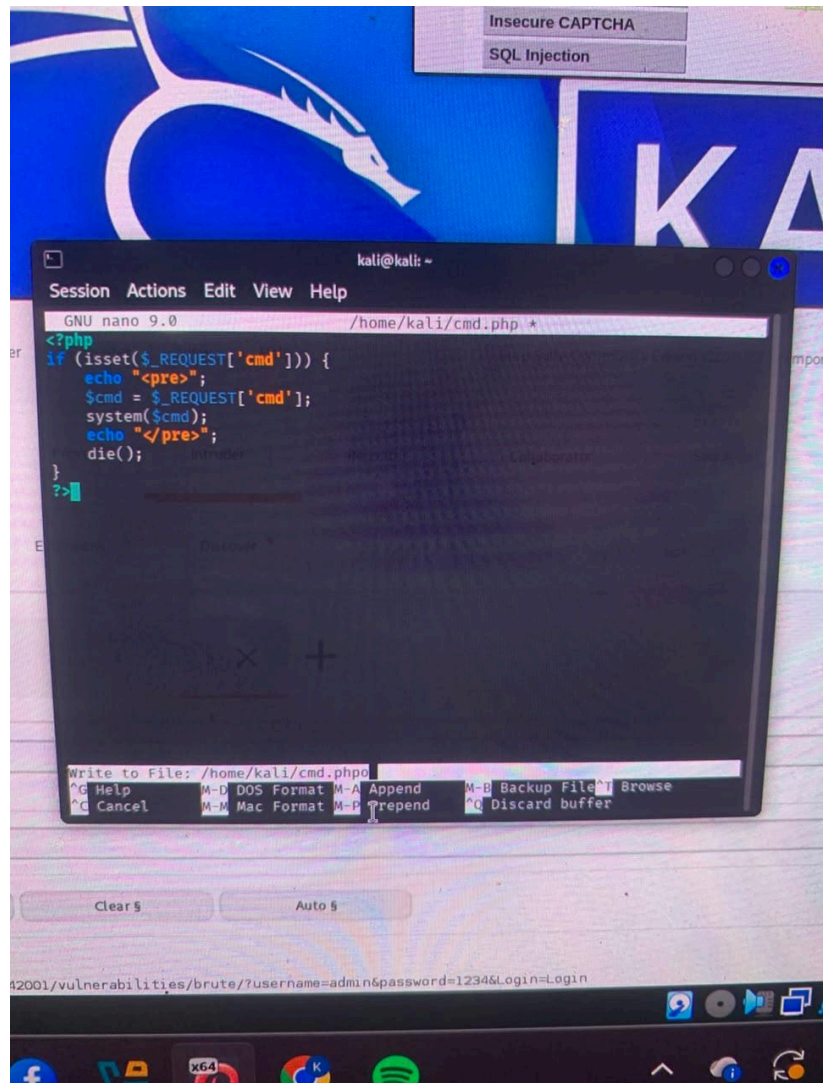
Tampilan konfirmasi **Password Changed** berwarna merah pada modul CSRF DVWA. Hal ini membuktikan bahwa eksekusi file eksploitasi csrf.html yang dijalankan via browser berhasil mengubah password akun

target secara otomatis tanpa konfirmasi manual dari pengguna.



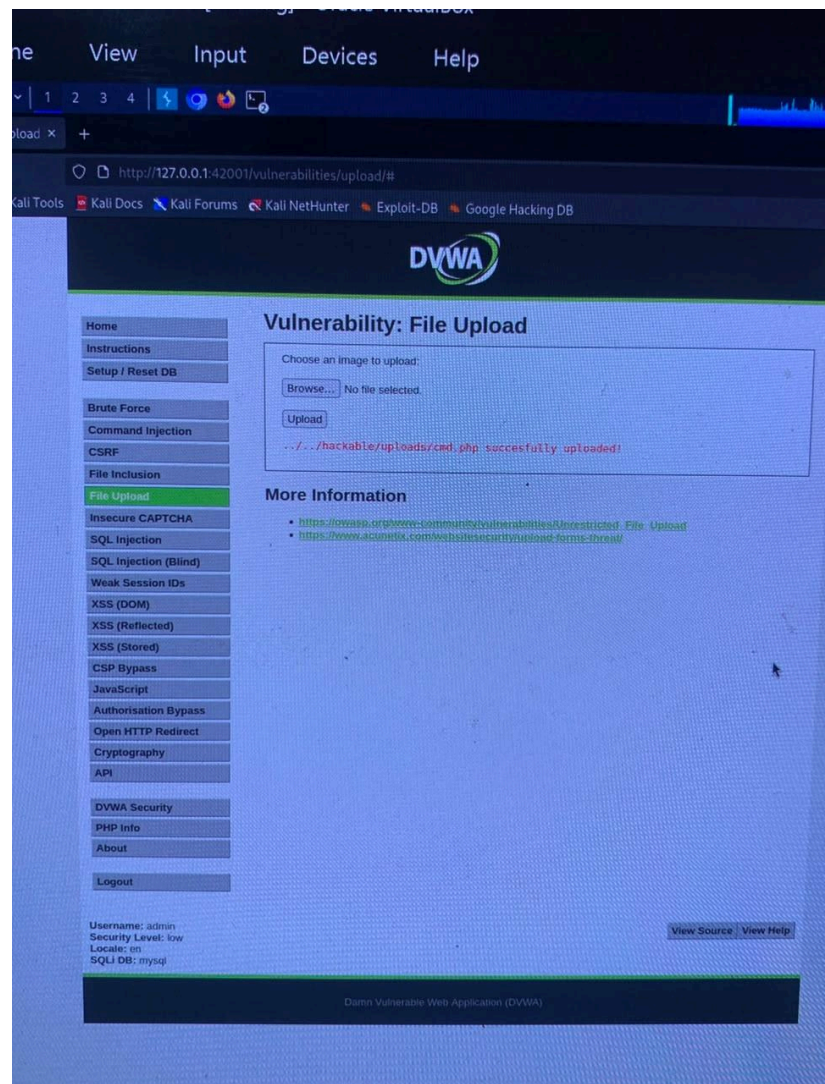
g. Pembuatan Script Web Shell PHP (cmd.php)

Tampilan pembuatan file skrip PHP cmd.php di terminal menggunakan teks editor **nano**. Skrip ini dikonfigurasi menggunakan fungsi `system($_REQUEST['cmd'])` untuk menerima serta mengeksekusi parameter perintah sistem.



h. Proses Pengungahan File Script pada Modul File Upload

Tampilan halaman modul *Vulnerability: File Upload* di DVWA setelah mengunggah file cmd.php. Hasilnya menampilkan pesan sukses `../../hackable/uploads/cmd.php successfully uploaded!` beserta lokasi direktori penyimpanannya.



i. Analisis Server & Pengecekan Service Web (Terminal)

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ curl -I http://127.0.0.1:42001/
HTTP/1.1 302 Found
Server: nginx/1.30.1
Date: Wed, 09 Sep 2026 12:40:09 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Set-Cookie: security=impossible; path=/; HttpOnly
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=b88bcfe70137e24efd31183a0dcc74fb; expires=Thu, 10 Sep 2026 12:40:09 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Location: login.php

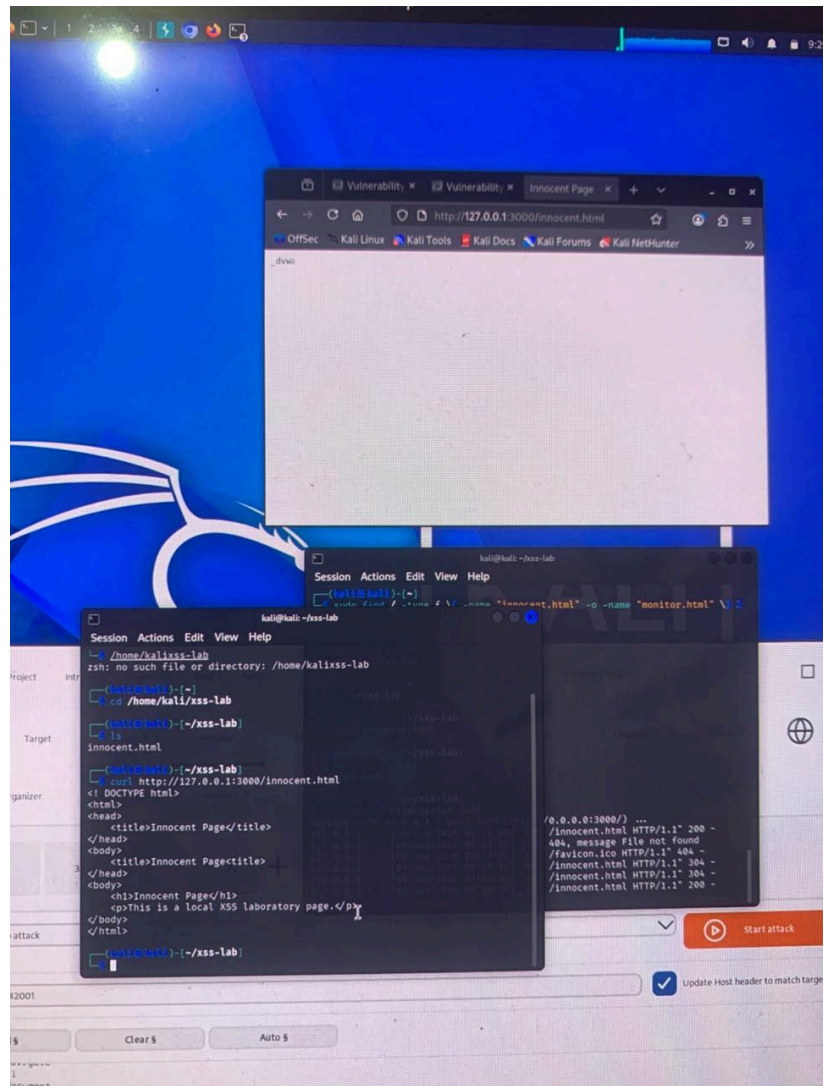
(kali@kali)-[~]
$ sudo ss -lntp | grep 42001
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
LISTEN 0      511          0.0.0.0:42001  0.0.0.0:*    users:((("nginx",pid=18749,fd=5),("nginx",pid=18748,fd=5),("nginx",pid=18747,fd=5)))

(kali@kali)-[~]
$

```

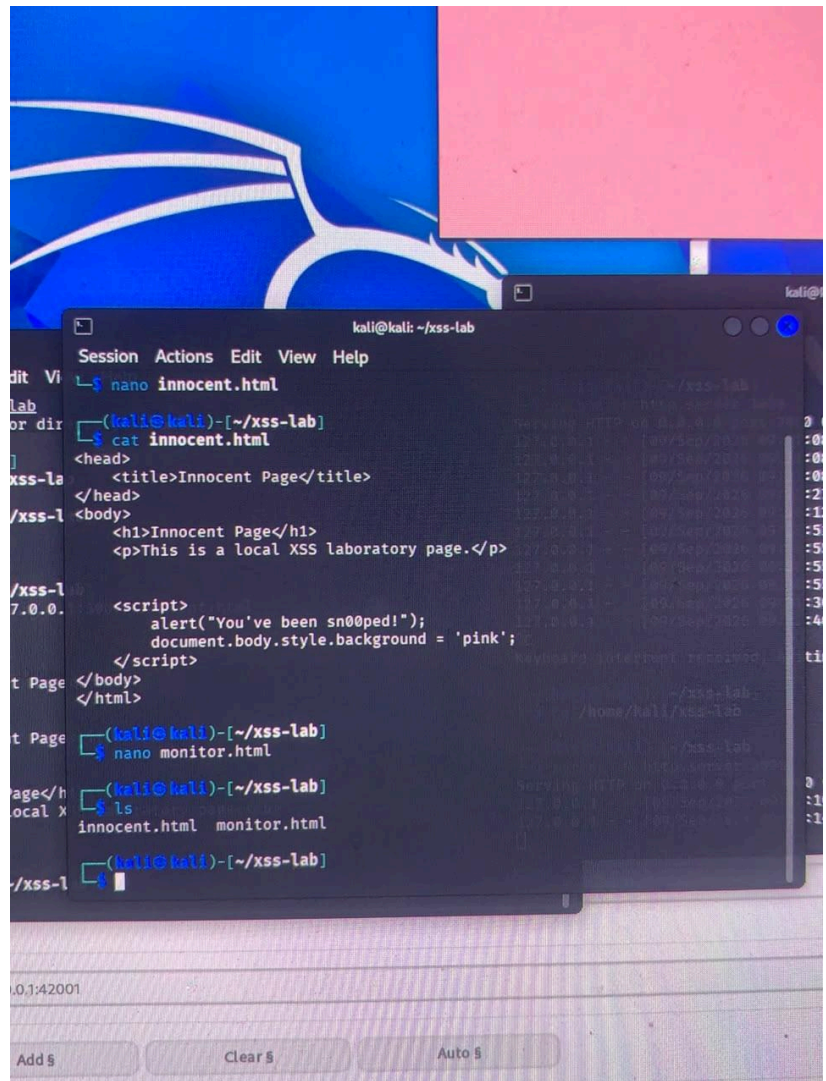
curl -I http://127.0.0.1:42001/ dan sudo ss -lntp | grep 42001. Dokumentasi ini membuktikan service web server Nginx/DVWA sedang aktif mendengarkan (*listening*) pada port 42001.

j. Pengujian Modul 7: XSS-Keylogger & Advanced Monitoring Lab



Tampilan pengujian lab XSS eksternal pada direktori `/home/kali/xss-lab`. Terminal menampilkan struktur kode `innocent.html` beserta *HTTP access log* pada server port 3000, sementara browser berhasil memuat halaman target `[http://127.0.0.1:3000/innocent.html](http://127.0.0.1:3000/innocent.html)`

k. Struktur Kode Payload XSS di `innocent.html`



```

kali@kali: ~/xss-lab
Session Actions Edit View Help
└─$ nano innocent.html
(kali@kali)-[~/xss-lab]
└─$ cat innocent.html
<head>
<title>Innocent Page</title>
</head>
<body>
<h1>Innocent Page</h1>
<p>This is a local XSS laboratory page.</p>
<script>
  alert("You've been sn00ped!");
  document.body.style.background = 'pink';
</script>
</body>
</html>
(kali@kali)-[~/xss-lab]
└─$ nano monitor.html
(kali@kali)-[~/xss-lab]
└─$ ls
innocent.html  monitor.html
(kali@kali)-[~/xss-lab]
└─$

```

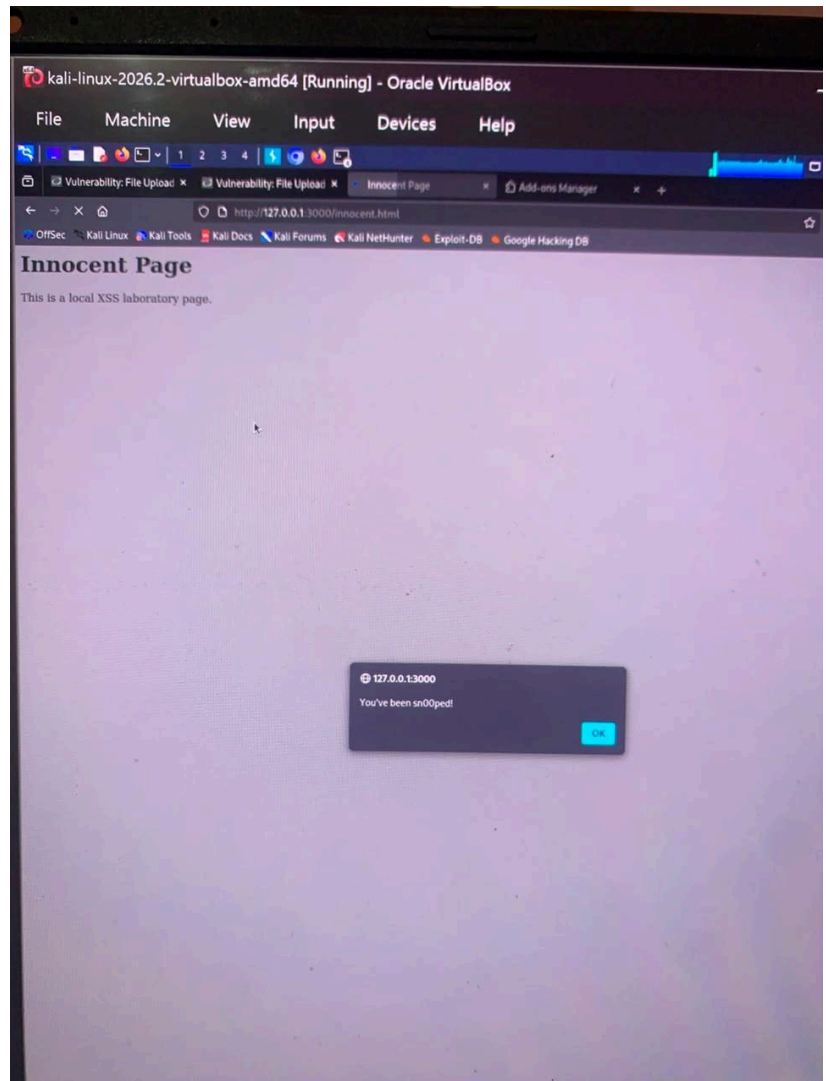
Tampilan terminal memperlihatkan kode innocent.html yang telah disisipi skrip JavaScript:

```

<script>
  alert("You've been sn00ped!");
  document.body.style.background = 'pink';
</script>

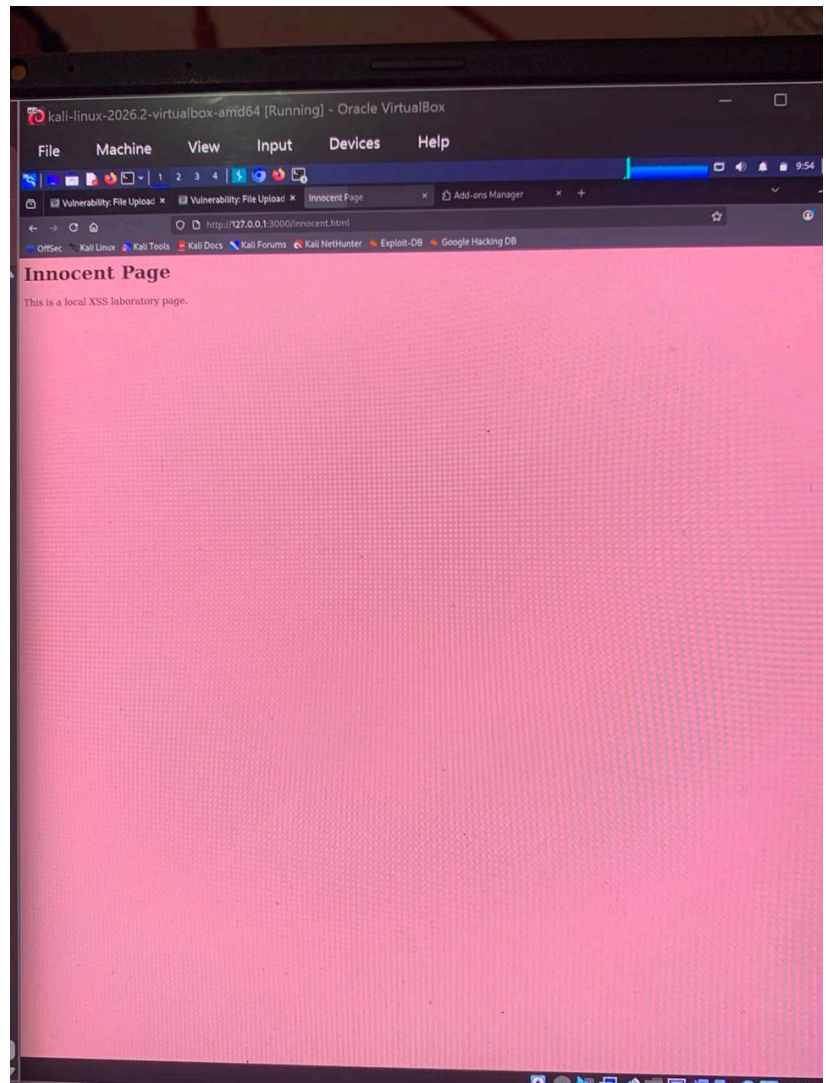
```

1. **Eksekusi Pop-up Alert "You've been sn00ped!"**



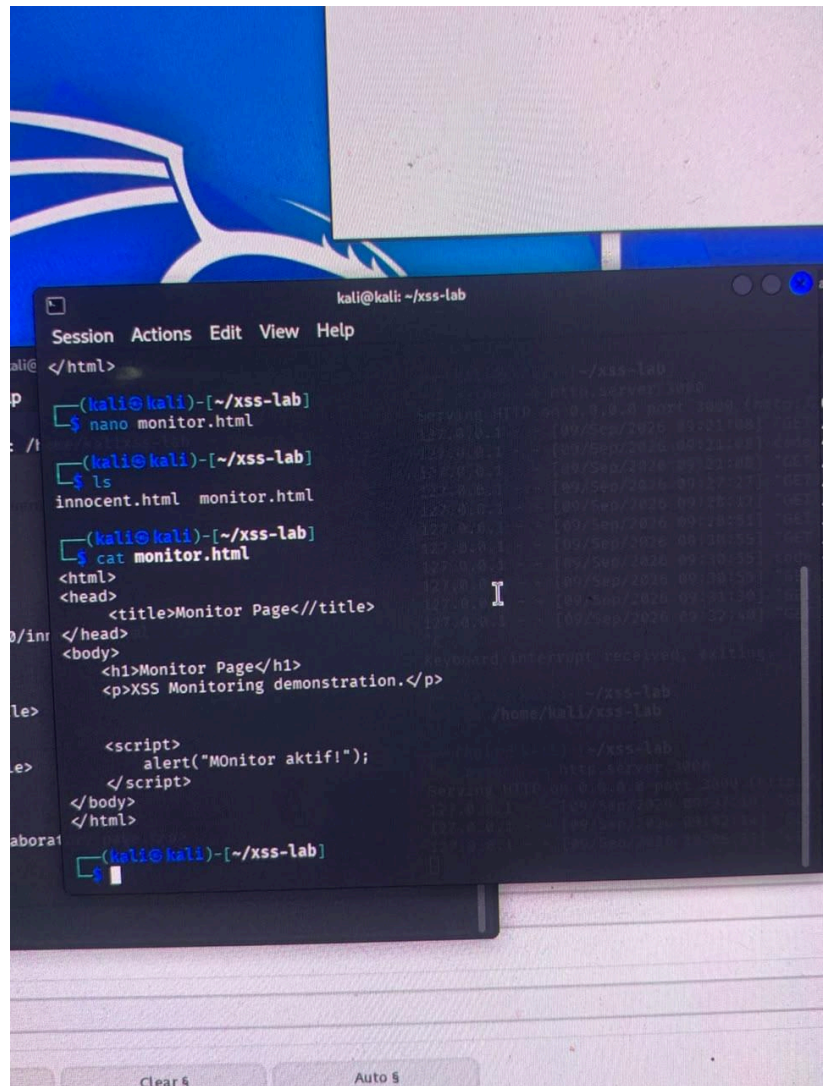
Tampilan browser saat memuat http://127.0.0.1:3000/innocent.html. Pemicuan skrip XSS berhasil mengeluarkan dialog pop-up peringatan dari jarak jauh.

m. Manipulasi DOM (Perubahan Background Menjadi Pink)



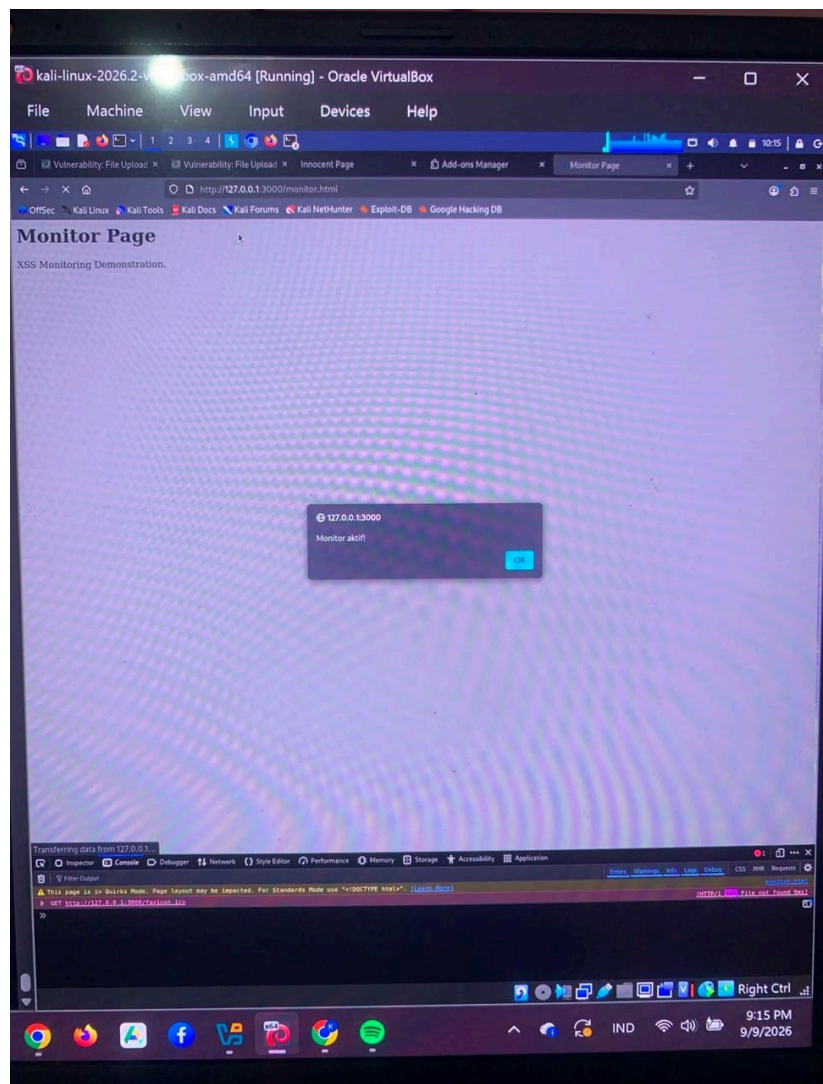
Tampilan antarmuka halaman innocent.html setelah tombol OK ditekan. Seluruh warna latar belakang (*background*) halaman berubah menjadi warna **pink** secara dinamis melalui eksekusi skrip `document.body.style.background = 'pink'`

n. Kode & Hasil Pengujian Halaman monitor.html



```
kali@kali: ~/xss-lab
Session Actions Edit View Help
</html>
(kali@kali)-[~/xss-lab]
$ nano monitor.html
(kali@kali)-[~/xss-lab]
$ ls
innocent.html  monitor.html
(kali@kali)-[~/xss-lab]
$ cat monitor.html
<html>
<head>
  <title>Monitor Page<\/title>
<\/head>
<body>
  <h1>Monitor Page<\/h1>
  <p>XSS Monitoring demonstration.<\/p>

  <script>
    alert("MOnitor aktif!");
  <\/script>
<\/body>
<\/html>
(kali@kali)-[~/xss-lab]
$
```

Tampilan pembuatan kode monitor.html berisi `alert("Monitor aktif!")` di terminal beserta bukti eksekusi pop-up-nya di browser saat memuat URL `[http://127.0.0.1:3000/monitor.html]` (`http://127.0.0.1:3000/monitor.html`)

2. Analisis Praktikum

Berdasarkan hasil praktikum di atas, dapat dilakukan analisis sebagai berikut:

- a. **SQL Injection & Command Injection:**
Kerentanan terjadi karena aplikasi tidak

melakukan *input validation* dan *sanitization*. Input pengguna diproses langsung oleh mesin basis data/shell OS, sehingga penyerang dapat memanipulasi kueri SQL maupun menjalankan perintah perintah sistem lokal secara sewenang-wenang.

- b. **XSS (DOM) & XSS-Keylogger:** Celah Cross-Site Scripting memicu eksekusi skrip JavaScript di sisi klien (*client-side*). Hal ini memungkinkan manipulasi elemen *Document Object Model* (DOM) seperti mengubah warna halaman, memicu pop-up pesan, hingga merekam aktivitas ketikan pengguna (*keylogging*) serta mencuri data cookie sesi.
- c. **Brute Force & CSRF:** Kelemahan pada manajemen autentikasi dan sesi. Brute Force mengeksplorasi ketiadaan pembatasan percobaan login (*rate limiting*) menggunakan Burp Suite Intruder. Sedangkan CSRF memanfaatkan cookie sesi yang aktif untuk memaksa browser mengeksekusi tindakan sensitif (seperti mengubah password) melalui permintaan HTTP buatan tanpa verifikasi token anti-CSRF.
- d. **Unrestricted File Upload:** Terjadi akibat ketiadaan pembatasan ekstensi file (*whitelist*) di server web. Server menerima file skrip eksekusi (seperti PHP) dan menyimpannya di folder publik, yang berpotensi memicu *Remote Code Execution* (RCE) jika dipanggil via URL.

BAB V

SIMPULAN

Berdasarkan penelitian yang telah dilakukan tentang autentikasi dan otorisasi, beberapa kesimpulan dapat diambil sebagai berikut:

1. **Akar Permasalahan:** Mayoritas celah keamanan aplikasi web bersumber dari ketiadaan validasi, sanitisasi, dan verifikasi *input/output* yang memadai di sisi server maupun klien..
2. **Efektivitas Tools Testing:** Penggunaan *tools* standar seperti Kali Linux dan Burp Suite terbukti sangat efektif untuk mengidentifikasi celah kerentanan secara terstruktur sebelum dieksploitasi oleh pihak tak berwenang..
3. **Penerapan Mitigasi Kunci:** Keamanan sistem web dapat ditingkatkan secara signifikan melalui penerapan *Prepared Statements* (anti-SQLi), *Output Encoding* (anti-XSS), *Anti-CSRF Tokens*, *Rate Limiting* (anti-Brute Force), serta pembatasan ketat ekstensi file di luar direktori web root..

DAFTAR PUSTAKA

Damn Vulnerable Web Application (DVWA). (2020). *DVWA documentation and security testing environment*. GitHub.
<https://github.com/digininja/DVWA>

OWASP Foundation. (2021). *OWASP Top 10:2021 - The ten most critical web application security risks*. Open Web Application Security Project. <https://owasp.org/Top10/>

Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). John Wiley & Sons.
<https://www.wiley.com/en-us/The+Web+Application+Hacker%27s+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472>

Sons.Acunetix. (2023). *Web application vulnerability report*. Invicti Security. <https://www.acunetix.com/vulnerability-report/>

Kementerian Komunikasi dan Digital RI. (2020). *Panduan teknis pengujian penetrasi sistem elektronik*. Kemkomdigi RI.
<https://layanan.komdigi.go.id/>

